

HBnB Evolution – Technical Documentation

Introduction

Purpose

This document provides the technical design and architecture for the HBnB Evolution application. It serves as a blueprint for the implementation phase by describing the system architecture, business logic, and interactions between components.

Project Overview

HBnB Evolution is a simplified Airbnb-style application that allows users to:

- Register and manage accounts
- Create and manage place listings
- Submit reviews for places
- Manage amenities associated with places

The application follows a layered architecture composed of:

1. Presentation Layer
2. Business Logic Layer
3. Persistence Layer

The Facade design pattern is used to simplify communication between layers and provide a unified interface to the business logic.

1. High-Level Architecture

High-Level Package Diagram



Architecture Overview

The system is organized into three layers:

Presentation Layer

Responsible for handling user interactions and API requests.

Components:

- API Endpoints
- User Service
- Place Service
- Review Service
- Amenity Service

Business Logic Layer

Contains the core domain models and business rules.

Components:

- User
- Place
- Review
- Amenity

Persistence Layer

Responsible for storing and retrieving information from the database.

Components:

- User Repository
- Place Repository
- Review Repository
- Amenity Repository

Facade Pattern

The Facade acts as a unified entry point to the Business Logic Layer.

Benefits:

- Simplifies communication between layers
- Reduces coupling

- Improves maintainability
- Centralizes business operations

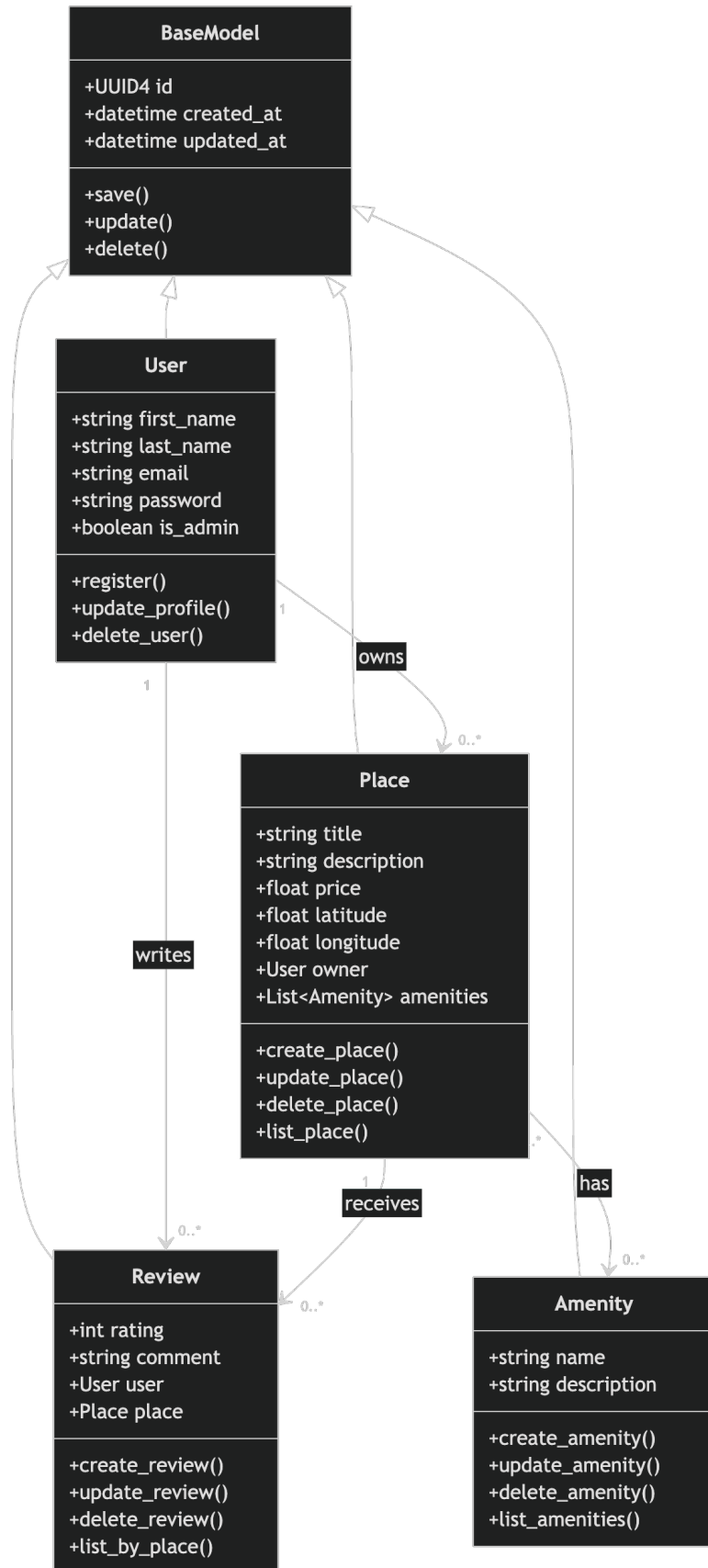
Request Flow

User → API → Facade → Business Logic → Persistence Layer → Database

Response follows the reverse path back to the user.

2. Business Logic Layer

Detailed Class Diagram



BaseModel

Base class inherited by all entities.

Attributes:

- id (UUID4)
- created_at
- updated_at

Methods:

- save()
- update()
- delete()

User

Represents a user of the application.

Attributes:

- first_name
- last_name
- email
- password
- is_admin

Methods:

- register()
- update_profile()
- delete_user()

Place

Represents a property listed on the platform.

Attributes:

- title
- description
- price
- latitude
- longitude
- owner

Methods:

- create_place()
- update_place()
- delete_place()

Review

Represents feedback left by users.

Attributes:

- rating
- comment
- user
- place

Methods:

- create_review()
- update_review()
- delete_review()

Amenity

Represents features available at a place.

Attributes:

- name
- description

Methods:

- create_amenity()
- update_amenity()
- delete_amenity()

Relationships

User → Place

One user can own multiple places.

User → Review

One user can write multiple reviews.

Place → Review

One place can receive multiple reviews.

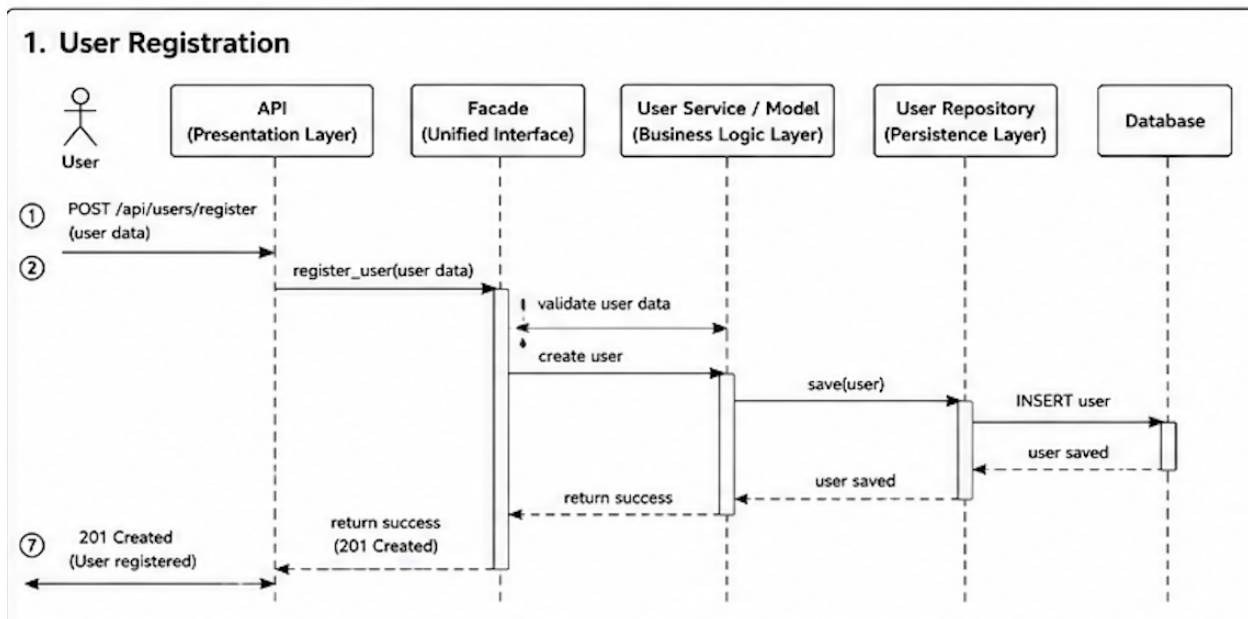
Place ↔ Amenity

Many-to-many relationship.

A place can contain many amenities, and an amenity can be associated with many places.

3. API Interaction Flow

3.1 User Registration



Description

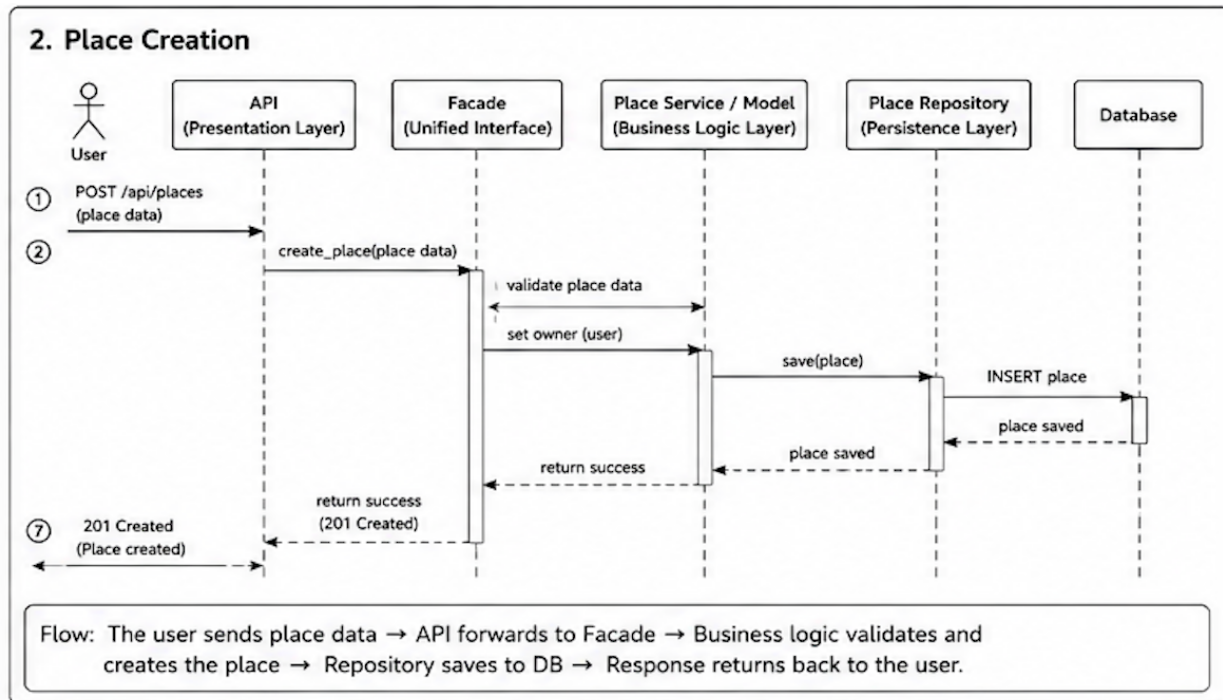
A new user creates an account.

Flow

1. User submits registration data.
2. API receives request.
3. Facade forwards request to User model.

4. User data is validated.
5. User repository stores information.
6. Database confirms creation.
7. Success response is returned.

3.2 Place Creation



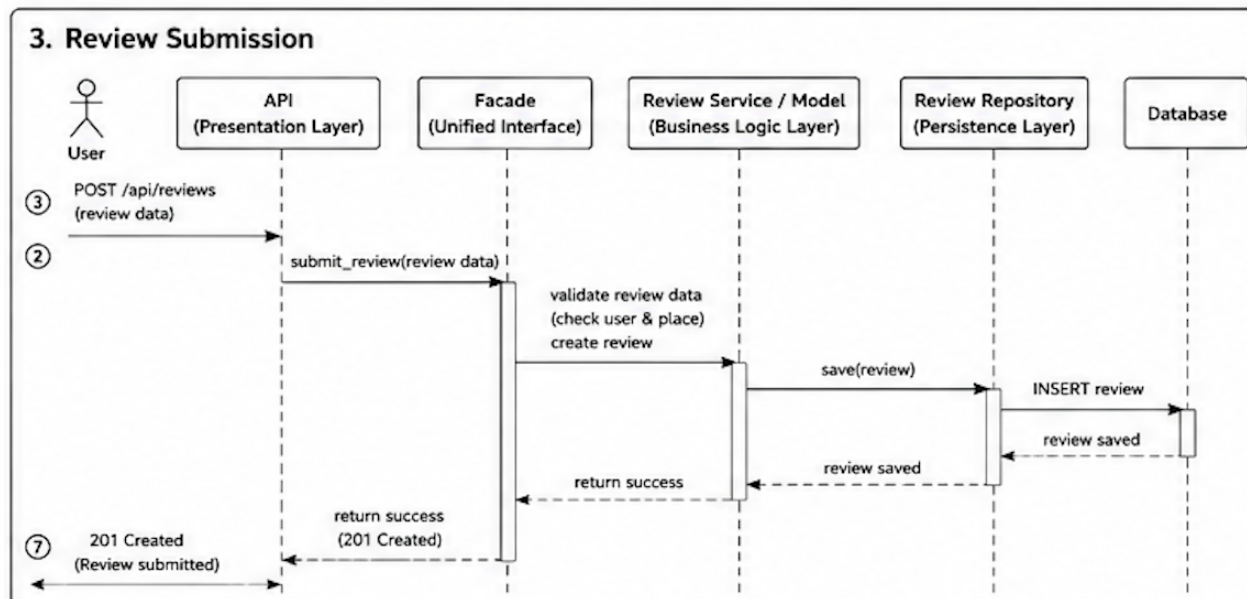
Description

A user creates a new place listing.

Flow

1. User submits place information.
2. API receives request.
3. Facade forwards request to Place model.
4. Place data is validated.
5. Place repository saves information.
6. Database confirms creation.
7. Success response is returned.

3.3 Review Submission



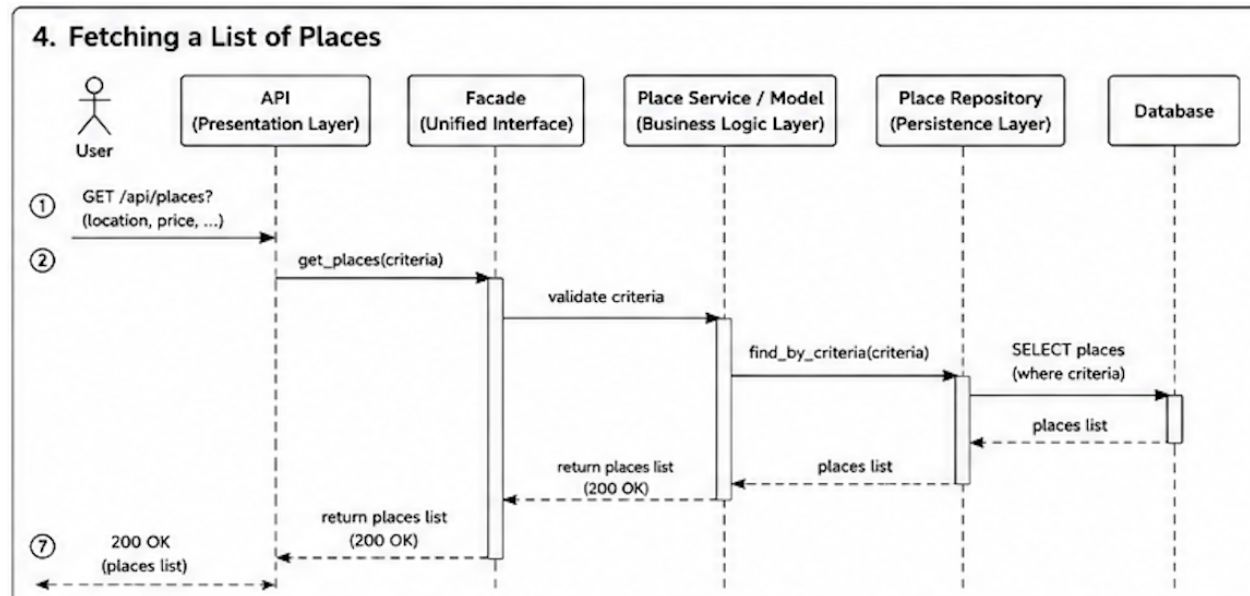
Description

A user submits a review for a place.

Flow

1. User submits review information.
2. API receives request.
3. Facade forwards request to Review model.
4. Review is validated.
5. Review repository saves data.
6. Database confirms creation.
7. Success response is returned.

3.4 Fetching a List of Places



Description

A user requests a list of places matching specific criteria.

Flow

1. User sends search request.
2. API receives request.
3. Facade processes criteria.
4. Repository queries database.
5. Database returns matching places.
6. Results are returned through the API.
7. User receives the list.

Conclusion

This document provides the architectural blueprint for the HBnB Evolution project.

The layered architecture promotes separation of concerns, maintainability, and scalability. The Facade pattern simplifies communication between layers, while the business logic layer encapsulates the core domain models and rules. The sequence diagrams demonstrate how user requests are processed across the system.

This documentation will serve as the foundation for future implementation phases of the HBnB application.